

AI / Agents

Rational agent: perceives environment + uses knowledge, history, goals to choose action maximizing performance.
Limited rationality: exact optimal action may be too expensive.
AI view: acting rationally $\neq$ acting human-like.

Uninformed Search

Alg	C	O	Key / Cost
BFS	Y	unit	shallowest; $T, S = O(b^d)$
UCS	Y	Y	min g ; $O(b^{C^*/\epsilon})$
DFS	finite m	N	deepest; $T = O(b^m), S = O(bm)$
DLS	$d \leq l$	N	cutoff l ; $T = O(b^l), S = O(bl)$
IDS	Y	unit	repeated DLS; $T = O(b^d), S = O(bd)$

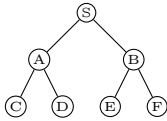
b : branch; d : optimal depth; m : max depth; ϵ : min step cost.
Trap: BFS optimal only for equal costs. UCS orders by total path cost. DFS may fail in infinite depth.

BFS / UCS / DFS Mini

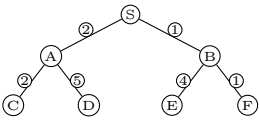
BFS: FIFO fringe, append children to end. UCS: priority queue by accumulated $g(n)$. DFS: stack/LIFO, low memory.

Mini: $S \rightarrow A = 2, S \rightarrow B = 1, A \rightarrow G = 2, B \rightarrow G = 10$. UCS expands B first, but returns S-A-G cost 4.

Search Tree Mini



BFS: S, A, B, C, D, E, F . DFS L→R: S, A, C, D, B, E, F .
DLS $l = 1$: S, A, B . DLS $l = 2$: S, A, C, D, B, E, F .
IDS: run DLS 0,1,2,... until goal found.



UCS: pop min path cost $g(n)$: S, B, A, F, C, E, D under shown costs.

Informed Search

Greedy best-first: minimizes $h(n)$ only. Fast if heuristic good; not optimal.

A*: minimizes total estimated path cost.

$$f(n) = g(n) + h(n)$$

g : cost so far. h : estimated remaining cost.
Admissible: $h(n) \leq h^*(n)$, never overestimates.
Non-admissible h : A^* no longer guarantees optimality, but may still find good solutions faster.
Consistent: $h(n) \leq c(n, n') + h(n')$.
Consistent $\Rightarrow$ admissible. Admissible $\nRightarrow$ consistent.
 A^* optimal if h admissible. Optimally efficient if h consistent.
Admissible checks h against the true remaining cost h^* ; $h(n) \leq h^*(n)$. Consistent checks every edge: $h(n) \leq c(n, n') + h(n')$. Consistent $\Rightarrow$ admissible, but not vice versa.
Dominance: h_2 dominates h_1 if $h_2(n) \geq h_1(n) \forall n$ and both admissible; A^* with h_2 expands fewer/equal nodes. More informed; closer to true cost h^* .
Relaxed problem: remove constraints; relaxed solution cost gives admissible h .
A* Mini
Edges: $S \rightarrow A = 2, S \rightarrow B = 1, A \rightarrow G = 4, B \rightarrow G = 10$;
 $h(A) = 3, h(B) = 1, h(G) = 0$.
 $f(A) = 2 + 3 = 5, f(B) = 1 + 1 = 2$. A^* expands B first, sees G cost 11, then expands A and returns S-A-G cost 6.
Trap: In A^* , goal generated $\neq$ goal selected. Return when goal is popped.

Local Search

Hill climbing: keep one current state; move to best neighbor. Not complete/optimal.
For minimization: accept child if $v(child) < v(current)$.
Problems: may fail even if solution exists because it keeps only local/current state; can get stuck in local optimum, plateau, or ridge.
Fix: random restart; sideways moves with visit limit; macro moves; limited lookahead.
Beam search: keep best k states per level. Low memory; not complete/optimal.
Trap: Beam width k is not branching factor b ; beam search is level-wise pruning, not k independent hill-climbers. Discarded states are gone. Hill climbing vs A^* : HC uses low memory/current state only, but not complete/optimal; A^* stores frontier and can be optimal with admissible h .

Simulated Annealing / GA

Simulated annealing: random neighbor; accept better always; accept worse with probability:

$$p = \exp\left(\frac{v(current) - v(next)}{T}\right)$$

for minimization. High T : exploration; low T : greedy.
Cooling too fast $\Rightarrow$ local optimum. Too slow $\Rightarrow$ inefficient.
Mini: $v(c) = 10, v(n) = 11, T = 2$. Worse move: $p = e^{-1/2} = 0.607$. If random $r = 0.4$, accept.
Genetic algorithm: population + fitness + selection + crossover + mutation.

$$P_i = \frac{f(S_i)}{\sum_j f(S_j)}$$

Selection gives uphill pressure; crossover/mutation gives exploration. Encoding matters.

Games

Minimax: 2-player, deterministic, perfect-info, zero-sum. MAX assumes MIN optimal.

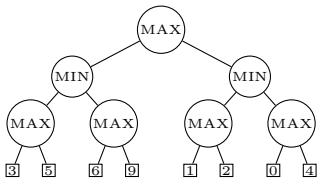
$$V(s) = \begin{cases} U(s), & \text{terminal} \\ \max_{s'} V(s'), & \text{MAX} \\ \min_{s'} V(s'), & \text{MIN} \end{cases}$$

Time $O(b^m)$, space $O(bm)$.
Mini: MAX choices: $A \rightarrow \{3, 5\}$, $B \rightarrow \{2, 9\}$. MIN backs $A=3$, $B=2$, MAX chooses A.
Alpha-beta: same result as minimax; skips irrelevant branches.
 α : best lower bound for MAX. β : best upper bound for MIN.
Prune when $\alpha \geq \beta$. Best $O(b^{m/2})$, worst $O(b^m)$. Ordering affects pruning amount, not minimax value.
Mini: Root MAX has left value 100, so $\alpha = 100$. At right MIN, first child 20 gives $\beta = 20$. Since $\beta \leq \alpha$, prune rest.
Expectiminimax: chance node:

$$V = \sum_i p_i V_i$$

Mini: Chance children 2,4 with $p = .5$: $V = 3$.

Game Tree Mini



Minimax: bottom-up; MAX takes max, MIN takes min. Left MIN=min(5, 9) = 5; right MIN=min(2, 4) = 2; root=max(5, 2) = 5.
 $\alpha\beta$ L→R: after left subtree, root $\alpha = 5$. In right MIN, first child gives $\beta = 2$. Since $\beta \leq \alpha$, prune remaining child (0, 4).
Pruned leaves cannot change ancestor choice because current α , β bounds already make that branch irrelevant.

ML Basics

Supervised: labelled data. Classif. = discrete label; regression = numeric value.
Unsupervised: no labels. Clustering = group similar examples.
Reinforcement: feedback/reward. **Association:** co-occurring patterns.
Train set: fit model. Valid. set: tune/prune/early stop. Test set: final comparison only.

Evaluation

$$Acc = \frac{TP + TN}{TP + TN + FP + FN}$$
$$Error = 1 - Acc$$
$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_1 = \frac{2PR}{P + R}$$

Precision: predicted positive, how often right? Recall: real positive, how many found?
Use F_1 when classes imbalanced or FP/FN both matter.
Train = fit model; validation = tune/prune/early stop; test = final unbiased comparison.
Stratification: preserve class proportions. **k-fold CV:** each fold test once; average metrics.
LOOCV: $k = n$; expensive; deterministic. **Paired t-test:** same splits; fold-wise differences.
Trap: Never tune on test set. High FN cost $\Rightarrow$ focus recall; high FP cost $\Rightarrow$ focus precision.

Clustering Concepts

Unsupervised partition: same-cluster similar, different-cluster dissimilar.
Good clustering = high cohesion + high separation.
Centroid = mean point, may not be real item. Medoid = actual central item.

$$c_i = \frac{1}{|K_i|} \sum_{x \in K_i} x$$

Agglomerative hierarchical: start singleton clusters; repeatedly merge closest clusters; update distance matrix after each merge.
Single link:

$$d(K_a, K_b) = \min_{x \in K_a, y \in K_b} d(x, y)$$

Complete link:

$$d(K_a, K_b) = \max_{x \in K_a, y \in K_b} d(x, y)$$

Average link: average pairwise distance.
Single link: chain effect. Complete link: compact clusters.
Davies-Bouldin:

$$DB = \frac{1}{k} \sum_i \max_{j \neq i} \frac{S_i + S_j}{d(c_i, c_j)}$$

Small DB = compact + far apart = good.
Mini: $K_1 = \{0, 2\}$, $K_2 = \{8, 10\}$. $c_1 = 1, c_2 = 9, S_1 = S_2 = 1$, $DB = .25$.

K-Means

Partitional, iterative, distance-based clustering.
Procedure: choose k centroids $\rightarrow$ assign to nearest $\rightarrow$ recompute centroids $\rightarrow$ repeat until stable.
Objective:

$$SSE = \sum_{i=1}^k \sum_{x \in K_i} d(x, c_i)^2$$

Time $O(tkmn)$; space $O((m+k)n)$.
Pros: simple, popular, efficient, usually fast.
Cons: need k ; init-sensitive; local minima; outliers; poor on non-spherical clusters.
k-means solve: for each epoch, assign all points to nearest current centroid $\rightarrow$ recompute each centroid as cluster mean $\rightarrow$ stop when clusters unchanged / max epochs.
1D: $d = |x - c|$; vector: usually Euclidean. Centroid = mean, not necessarily real point.
Trap: Use old centroids for all assignments in one epoch; do not update after each point.

Image Compression

1024^2 image, 2×2 blocks $\Rightarrow 512^2$ vectors, dimension 4:

$$rate = \frac{k \cdot 4 \cdot 8 + 512 \cdot 512 \log_2 k}{1024 \cdot 1024 \cdot 8}$$

NN Clustering / Hierarchical

Nearest Neighbor Clustering: incremental; process each item once.
If $d(new, K) \leq t$, join closest cluster; else new singleton. Usually single-link:

$$d(x, K) = \min_{y \in K} d(x, y)$$

Order-sensitive; threshold-sensitive; $O(n^2)$; chain effect.
Mini: $t = 2$: A starts; B dist 1 joins; C min 2 joins; D min 1 joins; E min 3 forms new cluster.
Hierarchical: dendrogram. No need k in advance.
Agglomerative = bottom-up merge. Divisive = top-down split.
Time $O(n^3)$ or $O(n^2 \log n)$; space $O(n^2)$.
Trap: Update distance matrix after each merge.

kNN

Lazy, instance-based. Training = store data.
kNN solve: given new example x , compare x with every training example; compute distance; sort by distance; take nearest k ; classification = majority vote, regression = average.
Nominal/Hamming: for each attribute, match = 0, mismatch = 1; Hamming distance = number of mismatched attrs.
Mini: New (*long, flat, spots, single*) vs (*long, flat, ruffles, single*): only texture differs $\Rightarrow d = 1$.
3NN: take 3 smallest distances, vote labels.
Classif. = majority vote. Regression = average.
Distances:

$$D_E = \sqrt{\sum_i (a_i - b_i)^2}$$
$$D_M = \sum_i |a_i - b_i|$$

Normalization:

$$a_i = \frac{v_i - \min v_i}{\max v_i - \min v_i}$$

Normalize because kNN is distance-based; large-scale attrs otherwise dominate distance.

Weighted kNN:

$$w_i = \frac{1}{D(x, x_i)}, \quad \hat{y} = \sum_i w_i y_i$$

Mixed attr: nominal match=0, mismatch=1; combine with numeric using chosen distance, e.g. $d = \sqrt{\sum(\Delta num)^2 + \sum(\Delta nom)^2}$.
Missing: worst-case / ignore depending setting.
Small $k \Rightarrow$ overfit/noisy. Large $k \Rightarrow$ smooth/biased.
Curse of dim.: distances become similar; irrelevant attrs hurt.

1R

One-rule classifier using one attribute.
1R solve: for each attr A, group by value v ; each group predicts majority class; group error = minority count; total error(A) = sum group errors; choose min-error attr. Classify new example using only that attr's rule.
1R rule tests one selected attribute only; each attribute value predicts its majority class.
Numeric: sort values; candidate breakpoints where adjacent class changes:

$$breakpoint = \frac{v_i + v_{i+1}}{2}$$

Mini: Age ≤ 30 : yes=2, no=3 $\Rightarrow$ predict no, error=2. Humidity example total error = $3 + 1 = 4$.

Naive Bayes

Choose class with largest posterior score:

$$\hat{c} = \arg \max_c P(c) \prod_j P(x_j | c)$$

Assumption: attrs conditionally independent given class. Often false: attrs may remain correlated even given class; still useful if posterior ranking works.
Bayes:

$$P(H|E) = \frac{P(E|H)P(H)}{P(E)}$$

Denominator common across classes, ignore for argmax.
NB solve: for each class c , start with prior $P(c) = \#c/N$; then look only at the new example's values. For each attr value x_j , count inside class c : $P(x_j | c) = \#(x_j, c) / \#c$. Multiply all: $score(c) = P(c) \prod_j P(x_j | c)$; choose class with max score. Numeric attr $\rightarrow$ Gaussian density.
Laplace:

$$P(x = v | c) = \frac{count(x = v, c) + 1}{count(c) + k}$$

m -estimate:

$$P(x = v_i | c) = \frac{n_i + mp_i}{n + m}$$

Gaussian numeric:

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-(x-\mu)^2/(2\sigma^2)}$$

Missing new attr: omit from product. Missing train value: omit from counts/denominator.
Mini: New (*long, flat, spots, single*). Safe: $P(S) = 5/10$ and conditionals $4/5, 3/5, 1/5, 4/5$; score $S = (5/10)(4/5)(3/5)(1/5)(4/5)$. Do same for other classes, choose max.
Trap: Use class-conditional probabilities, not unconditional probabilities.
Decision Trees
Choose split with highest information gain.
Entropy: $H = 0$ if pure; binary balanced $H = 1$; for k classes, $0 \leq H(S) \leq \log_2 k$, max when classes uniform.

$$H(S) = - \sum_i p_i \log_2 p_i$$

$$H(S|A) = \sum_v \frac{|S_v|}{|S|} H(S_v)$$

$$Gain(S, A) = H(S) - H(S|A)$$

IG solve: 1) Count total class labels in parent set S and compute parent $H(S)$.
2) For candidate attr A , list each value v of A as one branch S_v .
3) In each branch S_v , count final class labels inside that branch only; branch entropy uses class ratios inside branch: denominator = $|S_v|$, numerators = class counts in S_v .
4) Weight each branch entropy by $|S_v|/|S|$ add to get $H(S|A)$.
5) Compute $Gain(S, A) = H(S) - H(S|A)$.
6) Repeat for other candidate attrs; choose max gain.
Trap: DT does not compute $P(value|class)$ like NB. It computes entropy of class labels inside each value branch, e.g. colour=red branch has 2 good, 1 bad $\Rightarrow H(2/3, 1/3)$. Branch weights $|S_v|/|S|$.

are only for weighting. Pre-prune: max depth / min leaf / min gain. Risk: underfit. Post-prune: full tree + valid. set; replace subtree with leaf if valid. accuracy improves/same. Numeric attr: sort values; candidate thresholds at class-change midpoints. **Missing values** ignore missing attr/example, impute mean/mode, treat missing as separate value, or split/weight by observed branch probabilities. **Missing values in DT gain** assign branch probabilities from non-missing examples. Cost-sensitive split:

$$\frac{Gain(S,A)^2}{Cost(A)} \quad \text{or} \quad \frac{2Gain(S,A) - 1}{(Cost(A) + 1)^w}$$

Gain ratio:

$$SplitInfo = - \sum_i \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|}$$

$$GainRatio = \frac{Gain}{SplitInfo}$$

Gain ratio penalizes many-valued / ID-like attr:; reduces IG bias toward many splits. **Trap:** Do not maximize remaining entropy. Maximize entropy reduction. **Perceptron** Single-layer neural classifier; linear boundary.

$$net = w \cdot p + b = \sum_i w_i p_i + b$$

$$a = step(net) = \begin{cases} 1, & net \geq 0 \\ 0, & net < 0 \end{cases}$$

$$e = t - a, \quad w_i new = w_i old + ep, \quad b new = b old + e$$

Solve: each example in order: $n = w^T p + b \rightarrow a = step(n) \rightarrow e = t - a \rightarrow$ update w, b immediately. Course: $step(0) = 1$. Decision boundary:

$$w \cdot p + b = 0$$

Limitation: single perceptron learns only linear decision boundary; cannot solve XOR/non-linearly separable data. No bias $\Rightarrow$ boundary passes origin. **Mini:** $w = [0, 0], b = 0, p = [0, 1], t = 0. a = 1, e = -1, w = [0, -1], b = -1$. **Trap:** Perceptron learning rule not guaranteed for non-linearly separable data. **MLP / Backprop** MLP with hidden layer + nonlinear differentiable activation forms nonlinear boundary. One hidden layer is universal approximator, but theorem does not tell how to find weights. **Forward:** for each neuron, $n = w^T x + b, a = f(n)$. FC params $= \sum_l n_l n_{l+1} + \sum_{l \geq 1} n_l$, where n_0 is input layer and later layers have biases. Sigmoid:

$$f(x) = \frac{1}{1 + e^{-x}}, \quad f'(x) = f(x)(1 - f(x))$$

Tanh derivative:

$$f'(x) = 1 - a^2$$

SSE:

$$E = \frac{1}{2} \sum_i (d_i - o_i)^2$$

Generic update:

$$\Delta w_{pq} = \eta \delta_q o_p, \quad w_{pq} \leftarrow w_{pq} + \Delta w_{pq}$$

Output neuron:

$$\delta_q = (d_q - o_q) f'(net_q)$$

Sigmoid output:

$$\delta_q = (d_q - o_q) o_q (1 - o_q)$$

Hidden neuron:

$$\delta_q = f'(net_q) \sum_i w_{qi} \delta_i$$

Sigmoid hidden:

$$\delta_q = o_q (1 - o_q) \sum_i w_{qi} \delta_i$$

Momentum:

$$\Delta w_{pq}(t) = \eta \delta_q o_p + \mu \Delta w_{pq}(t - 1)$$

η : learning rate; $\mu \in [0, 1)$: momentum.

Backprop Procedure / Issues **Backprop solve:** forward $net \rightarrow o$; compute output δ ; compute hidden δ from downstream old $w\delta$; then update all weights. Do not round early. Early stopping: monitor validation error; stop when valid. error stops improving/rises while train error keeps falling. **Input neurons:** numeric directly; nominal via one-hot. **Nominal inputs:** use one-hot; input count = total attribute values. Weight dim = input count; bias is extra parameter. Output: binary $\rightarrow 1$ output; k classes $\rightarrow k$ outputs. Often use .9/.1 instead of 1/0 to avoid flat gradient. **NN can do classification or regression/rating;** output can be continuous, not only binary. Hidden heuristic: roughly $(n_{in} + n_{out})/2$, then tune. Too few hidden units $\Rightarrow$ underfit. Too many $\Rightarrow$ overfit/slow. **LR** too small $\Rightarrow$ slow. LR too large $\Rightarrow$ oscillate/diverge. Overfit causes: small data, noise, too many trainable params. Fix: early stopping, different init, adaptive LR, dropout. **Local minima:** gradient descent may find local, not global, minimum; depends on initialization. **Vanishing gradient:** lower layers get tiny updates. **Trap:** Backprop needs differentiable activation. Hidden neurons have no direct target values. **Deep / Autoencoder / CNN** **Deep:** multiple hidden layers; learns hierarchical features. **Autoencoder:** target=input, $y = x$; bottleneck/narrow hidden layer learns compressed representation. **CNN:** image/grid data; local connectivity reduces params; weight sharing applies same learned filter across locations $\rightarrow$ feature map. Convolution:

$$feature = \sum_{i,j} x_{ij} w_{ij}$$

Max pooling:

$$pool = \max(x_1, \dots, x_r)$$

Pooling reduces dimension and gives shift invariance. Dropout: random deactivation during training; reduces overfit. **Trap:** CNN filter weights are learned and shared, not separate per pixel/location. **SVM** SVM objective: maximize margin between separating hyperplane and closest points / SVs.

$$w \cdot x + b = 0$$
$$w \cdot x + b = \pm 1$$
$$margin = \frac{2}{\|w\|}$$

Hard margin:

$$\min \frac{1}{2} \|w\|^2$$
$$s.t. \quad y_i(w \cdot x_i + b) \geq 1$$

Dual / classify:

$$w = \sum_i \lambda_i y_i x_i$$
$$f(z) = \sum_i \lambda_i y_i (x_i \cdot z) + b$$
$$\hat{y} = \text{sign}(f(z))$$

Soft margin:

$$0 \leq \lambda_i \leq C$$

Large C : fewer train errors, narrower margin. Small C : wider margin, more violations.

Kernel:

$$K(u, v) = \Phi(u) \cdot \Phi(v)$$
$$K(x, y) = (x \cdot y + 1)^P$$
$$K(x, y) = e^{-\|x - y\|^2 / (2\sigma^2)}$$

Kernel trick: compute dot product in feature space without explicit mapping. Linear separator in feature space = nonlinear boundary in original space. **Mini:** $w = (1, 1), z = (2, 1), b = 0: f = 3 > 0 \Rightarrow +1$. **Trap:** Not all points define boundary; SVs do. **SVM checks:** maximizes margin; $\lambda_i > 0$ means support vector. Training dot products: training vectors with training vectors. Testing: support vectors with new example. **Ensemble checks:** Bagging works for classification and regression. Boosting often helps but does not guarantee improvement. **SVM / Perceptron / NN** Perceptron: simple, efficient, linear only. Backprop NN: nonlinear, expressive, but many params, slow, local minima, architecture/LR sensitive. SVM: margin + kernels, strong generalization, but kernel/C tuning and train cost matter. Linear SVM: optimal linear boundary in original space. Nonlinear SVM: optimal linear boundary in feature space. Multi-class SVM: decompose into binary classifiers, e.g. one-vs-one gives $k(k - 1)/2$ classifiers.

Ensembles Good ensemble = individual accuracy + error diversity. Majority vote:

$$\hat{y} = mode(h_1(x), \dots, h_M(x))$$

Weighted vote:

$$\hat{y} = \arg \max_c \sum_m \alpha_m 1[h_m(x) = c]$$

Diversity		Train	Vote
Bag	bootstrap	parallel	equal
Boost	hard cases	seq.	weighted
RF	boot. + features	parallel	equal

Bagging reduces variance. Boosting focuses on difficult cases. RF lowers tree correlation. **Trap:** Many identical classifiers do not help. **Bagging** Bootstrap aggregating. Sample n examples with replacement for reduces variance; train one classifier per sample; majority vote / average. About 63% of original examples appear in a bootstrap sample. Regression:

$$\hat{y} = \frac{1}{M} \sum_{m=1}^M h_m(x)$$

Useful for unstable learners e.g. DT. Easy to parallelize. Less useful for stable learners. **Mini:** Original IDs 1,2,3,4,5. Bootstrap sample: 2,5,2,1,4. **Trap:** Bagging samples with replacement and uses equal voting. **Boosting / AdaBoost** Sequential learners; later classifiers focus on examples earlier classifiers got wrong. Weighted error:

$$\epsilon_t = \sum_i p_i e_i$$

$$\beta_t = \frac{\epsilon_t}{1 - \epsilon_t}$$

Correct examples:

$$p_i \leftarrow p_i \beta_t$$

then normalize. Classifier vote weight:

$$\alpha_t = \log \frac{1}{\beta_t}$$

Mini: 10 examples weight .1. Classifier gets 3 wrong: $\epsilon = .3, \beta = .3/.7 = .429$. Correct examples downweighted; wrong examples relatively upweighted. **Cons:** sensitive to noisy labels/outliers; sequential not parallel. **Trap:** Final boosting vote is weighted, not equal. **Boosting solve:** ϵ = sum weights of misclassified; $\beta = \epsilon/(1 - \epsilon)$; correct examples $\propto \beta$, wrong unchanged; normalize all. **Random Forest** RF = bagging + random feature selection + unpruned DTs. For each tree: bootstrap sample; grow DT without pruning; at each split choose k random features; select best among k using IG; majority vote. Rule:

$$k \approx \log_2 m$$

m = total features. Performance depends on high tree strength and low correlation. Increasing k : strength $\uparrow$, correlation $\uparrow$. **Pros:** often better than single DT; resists overfit; faster than AdaBoost with similar accuracy. **Trap:** Standard RF trees are not pruned; do not use all features at every split. **Probability Basics** Conditional:

$$P(A|B) = \frac{P(A, B)}{P(B)}$$

Product:

$$P(A, B) = P(A|B)P(B) = P(B|A)P(A)$$

Independence:

$$P(A, B) = P(A)P(B)$$

Disjunction:

$$P(A \vee B) = P(A) + P(B) - P(A \wedge B)$$

Bayes:

$$P(H|E) = \frac{P(E|H)P(H)}{P(E)}$$

Total probability:

$$P(X) = \sum_i P(X|Y_i)P(Y_i)$$

Chain rule:

$$P(x_1, \dots, x_n) = \prod_i P(x_i | x_1, \dots, x_{i-1})$$

Full joint solve: add rows where event is true. $P(A)$: sum rows with A ; $P(A, B)$: sum rows with both; $P(A|B) = P(A, B)/P(B)$. For scores, normalize by total sum. Normalize:

$$P(X|e) = \alpha P(X, e)$$

Mini: $P(M) = 1/50000, P(S|M) = .5, P(S) = 1/20. P(M|S) = .5(1/50000)/(1/20) = .0002$. **Trap:** Do not reverse $P(A|B)$ and $P(B|A)$. **Bayesian Networks** BN = directed acyclic graph + CPTs. Joint = product of local CPT terms:

$$P(x_1, \dots, x_n) = \prod_i P(x_i \mid Parents(x_i))$$

BN factor solve: for each node, look at graph arrows; root node $\rightarrow P(root)$, non-root $\rightarrow P(node \mid parents)$. Multiply all node factors. **Mini:** $F \rightarrow T, F \rightarrow S: P(F, T, S) = P(F)P(T \mid F)P(S \mid F)$. **BN posterior solve:** for $P(Q \mid e)$, make one unnormalised score for each value of query Q . Each score = joint product with evidence fixed. Sum over hidden vars if any. Then normalize:

$$P(Q = q \mid e) = \frac{score(q)}{\sum_{q'} score(q')}$$

Mini: $P(F \mid T+, S): score(F) = P(F)P(T+ \mid F)P(S \mid F)$; $score(\neg F) = P(\neg F)P(T+ \mid \neg F)P(S \mid \neg F)$; normalize. **Hidden vars:** if a variable is not query/evidence, sum over all values:

$$score(q) = \sum_h P(q, e, h)$$

Trap: Evidence vars are fixed. Query vars get one score per value. Hidden vars are summed out. Always normalize. Cond. independence: node independent of non-descendants given parents. Common cause: $F \rightarrow T, F \rightarrow S$. Children are conditionally independent given parent F : $P(T, S \mid F) = P(T \mid F)P(S \mid F)$, but not necessarily marginally independent. Markov blanket = parents + children + children's parents. Naive Bayes = special BN: class node parent of all features. **Applied Choice** Labelled+interpretable: DT; nominal attr: prune for overfit. Fast/count baseline: NB; easy update, but cond. independence assumption. No-train similarity: kNN; complex boundary, but slow query + scale-sensitive. Unlabelled grouping: k-means if k known/spherical; hierarchical if dendrogram/no k . Images/grid: CNN; local receptive fields + shared filters reduce params. Nonlinear/high-dim: SVM kernel or NN; SVM margin, NN expressive but tuning/overfit. Better DT accuracy: RF reduces variance/correlation; boosting focuses hard cases but noise-sensitive. Rating/regression: NN continuous output, not only binary classes. **Common Traps** A*/UCS stop when goal selected, not generated. Greedy uses h ; A* uses $g + h$; UCS uses g . Admissible $\neq$ consistent; consistent $\Rightarrow$ admissible. Precision denom.=predicted positives; recall denom.=actual positives. High FN cost $\Rightarrow$ recall. Valid. for tuning/pruning/early stop; test for final evaluation. kNN/K-means distance-based: normalize; nominal Hamming counts mismatches. NB uses class-conditional $P(x_j|c)$; DT IG uses weighted child entropy. Perceptron linear/XOR fail; backprop needs differentiable activation; GD local not global. SVM boundary by SVs; bagging bootstrap/equal vote; boosting sequential/weighted. BN: evidence fixed, hidden summed, scores normalized. CM: TP=actual+ /pred+; FN=actual+ /pred-; FP=actual- /pred+; TN=actual- /pred-. High FN cost $\rightarrow$ recall; high FP cost $\rightarrow$ precision.